

WEBPROGRAMOZÁS

CANVAS, ANIMÁCIÓK, API-K

Horváth Győző

egyetemi docens

horvath.gyozo@inf.elte.hu

1117 Budapest, Pázmány Péter sétány 1/c., 2.408

Tel: (1) 372-2500/8469

ISMÉTLÉS

ISMÉTLÉS

- JavaScript nyelvi elemei
- DOM programozás
- Eseménykezelés részletei
- Kódszervezés
- Időzítők
- Űrlapok, képek, táblázatok
- JavaScript beépített objektumai



CANVAS API

GRAFIKUS LEHETŐSÉGEK A BÖNGÉSZŐBEN

- Képfájlok
- **CSS formázás**
- **(Inline) SVG (vektorgrafika)**
- **Canvas (raszteres grafika)**

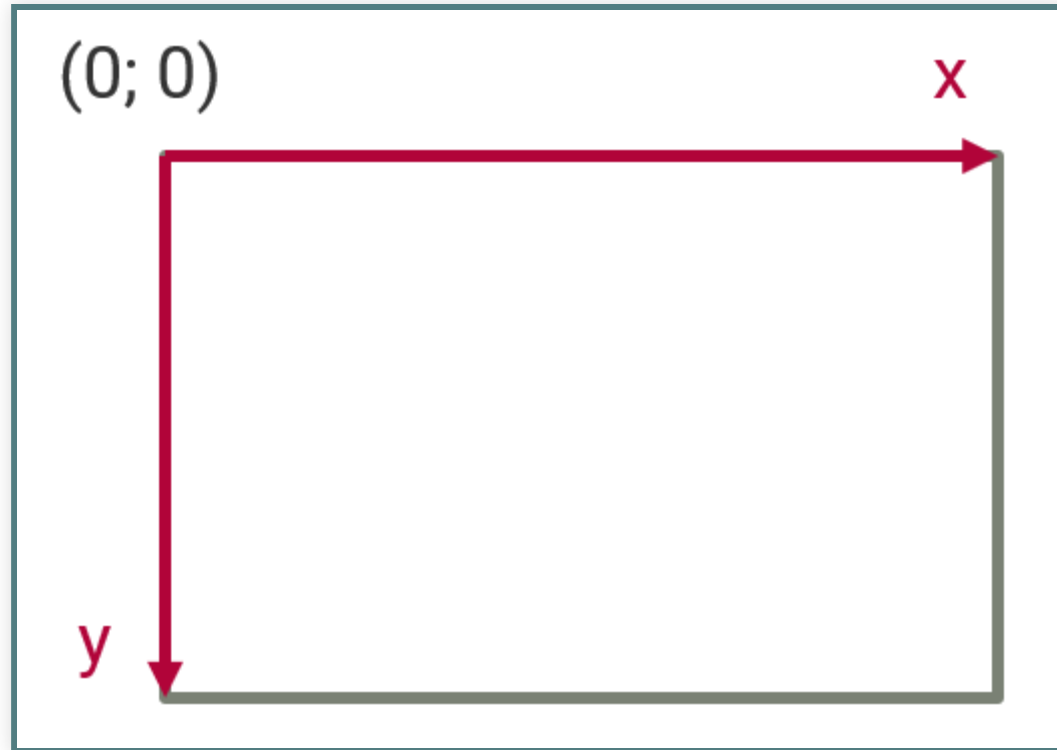
CANVAS

```
<canvas width="200" height="200"></canvas>
```

```
// Canvas elem elérése  
const canvas = document.querySelector('canvas');  
// 2D rajzolási "kontextus" elérése  
const context = canvas.getContext('2d');
```

Referencia

RAJZOLÁS A SZÁMÍTÓGÉPEN



Koordinátarendszer

RAJZOLÁS ALAKZATOKKAL, KÉPEKKEL

A `CanvasRenderingContext2D` műveletei:

- Szöveg (`fillText`, `strokeText`)
- Téglalap (`fillRect`, `strokeRect`)
- Kép (`drawImage`)
 - Forrás: kép, canvas, video
 - `drawImage(image, x, y)`
 - `drawImage(image, x, y, width, height)`
 - `drawImage(image, sx, sy, sWidth, sHeight, dx, dy, dWidth, dHeight)`
- Törlés (`clearRect`)

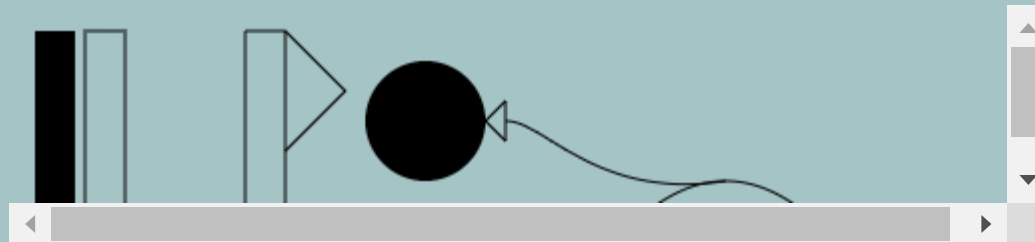
Referencia

RAJZOLÁS ÚTVONALAKKAL

- Útvonal kezdete (`beginPath`)
- Útvonal lezárása (`closePath`) !
- Útvonal kitöltése (`fill`)
- Útvonal megrajzolása (`stroke`)
- Téglalap (`rect`)
- Ellipszis (`ellipse`)
- Körív (`arc`)
- “Toll” mozgatása (`moveTo`)
- Vonal (`lineTo`)
- Bezier-görbe (`bezierCurveTo`)
- Kvadratikus görbe (`quadraticCurveTo`)

RAJZOLÁS ÚTVONALAKKAL

```
context.fillRect(5, 5, 20, 100);  
context.strokeRect(30, 5, 20, 100);  
  
context.beginPath();  
context.rect(110, 5, 20, 100);  
context.moveTo(130, 5);  
context.lineTo(160, 35);  
context.lineTo(130, 65);  
context.stroke();  
  
context.beginPath();  
context.arc(200, 50, 30, 0, 2 * Math.PI);  
context.fill();  
  
context.beginPath();  
context.moveTo(5, 200);  
context.quadraticCurveTo(55, 100, 105, 200);
```



BEÁLLÍTÁSOK

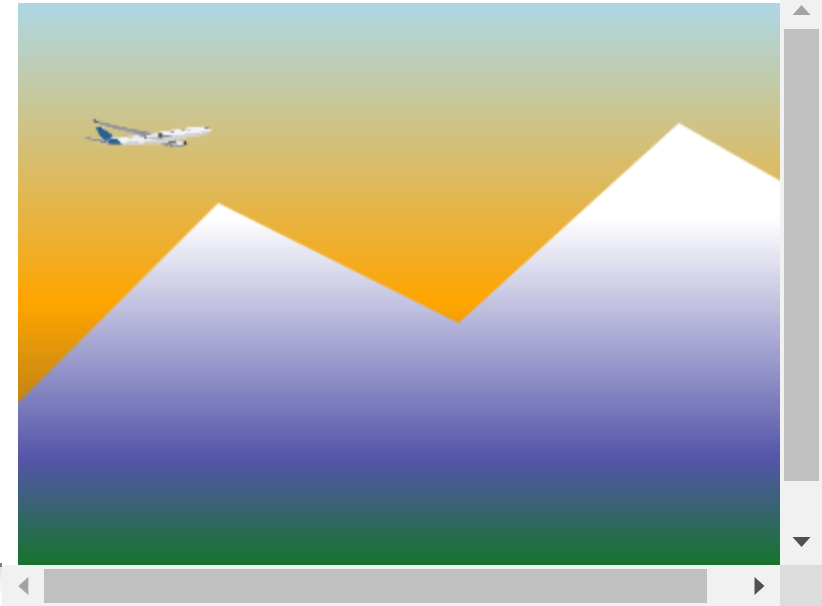
- Kitöltés színe/mintázata (`fillStyle`)
- Vonal színe (`strokeStyle`)
- Vonal vastagsága (`lineWidth`)
- Vonalak vége (`lineCap`)
- Vonalak illesztése (`lineJoin`)
- Áttetszőség (`globalAlpha`)
- Egyesítés és vágás

PÉLDA

```
const plane = document.createElement('img')
plane.src = 'plane.png'
plane.addEventListener('load', draw)
function draw() {
  const gradSky = ctx.createLinearGradient(200, 0, 200, 300)
  gradSky.addColorStop(0, 'lightblue')
  gradSky.addColorStop(0.5, 'orange')
  gradSky.addColorStop(1, '#444444')

  const gradMountain = ctx.createLinearGradient(0, 300, 400, 300)
  gradMountain.addColorStop(0, 'white')
  gradMountain.addColorStop(0.2, 'white')
  gradMountain.addColorStop(0.7, '#555555')
  gradMountain.addColorStop(1, 'green')

  ctx.fillStyle = gradSky
  ctx.fillRect(0, 0, 400, 300)
```

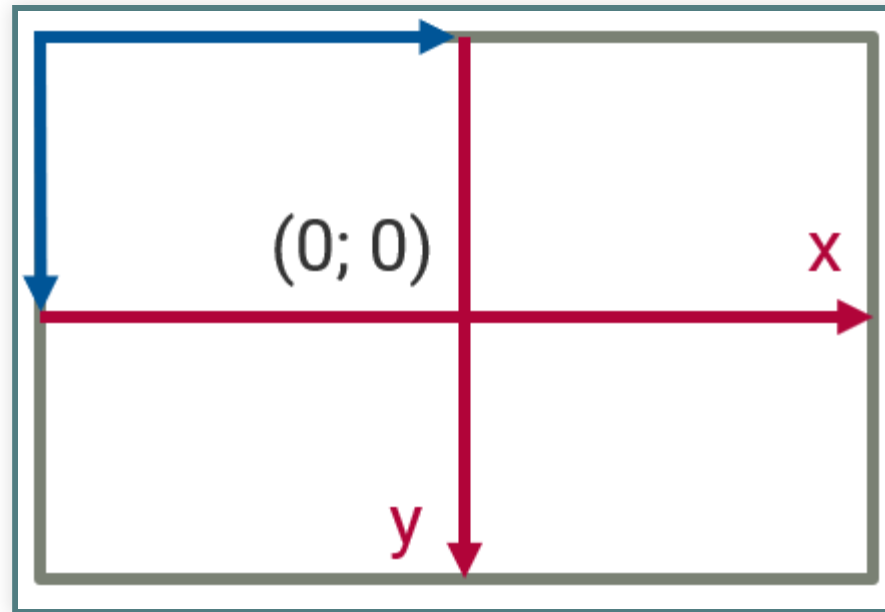


TRANSZFORMÁCIÓK

- Műveletek
 - Forgatás (`rotate`)
 - Méretezés (`scale`)
 - Eltolás (`translate`)
 - Összetett (`transform`)
- Állapotkezelés
 - `save`
 - `restore`

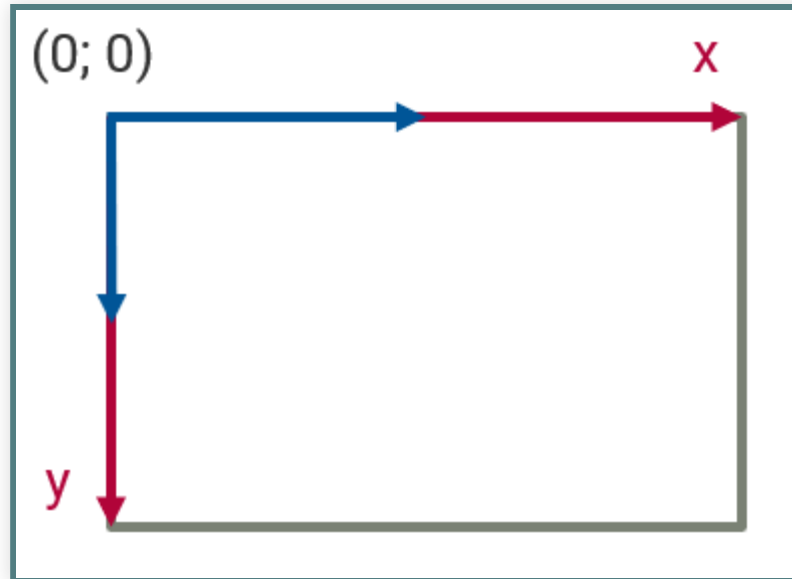
ELTOLÁS

```
context.translate(100, 100);
```



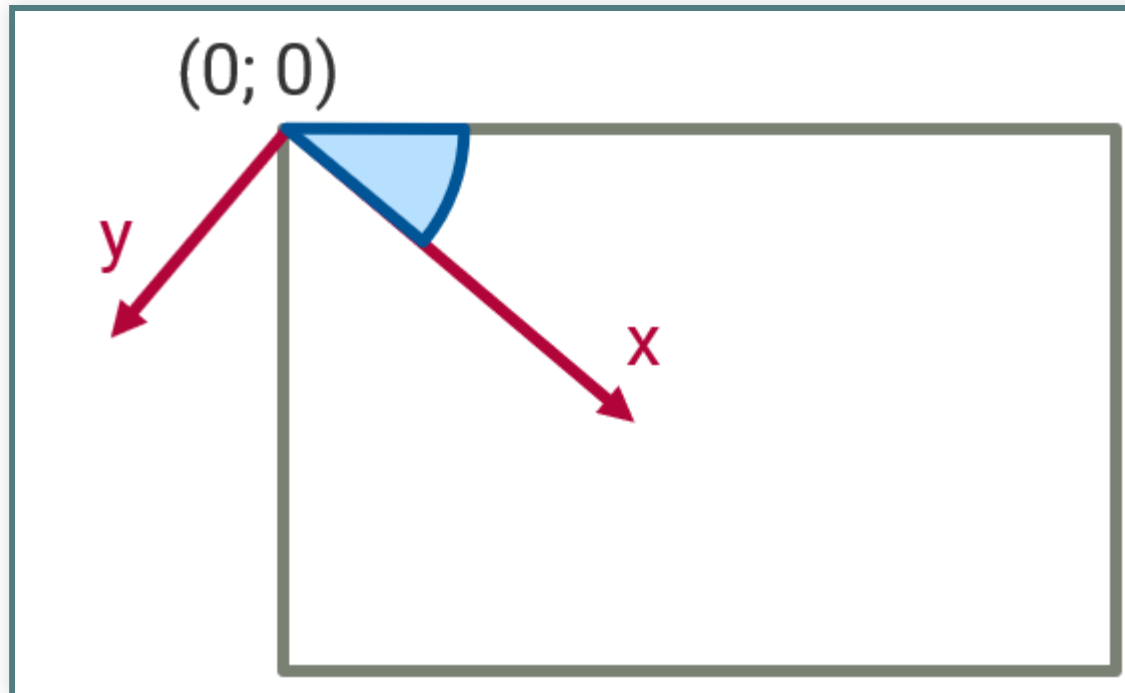
MÉRETEZÉS

```
context.scale(2, 2);
```



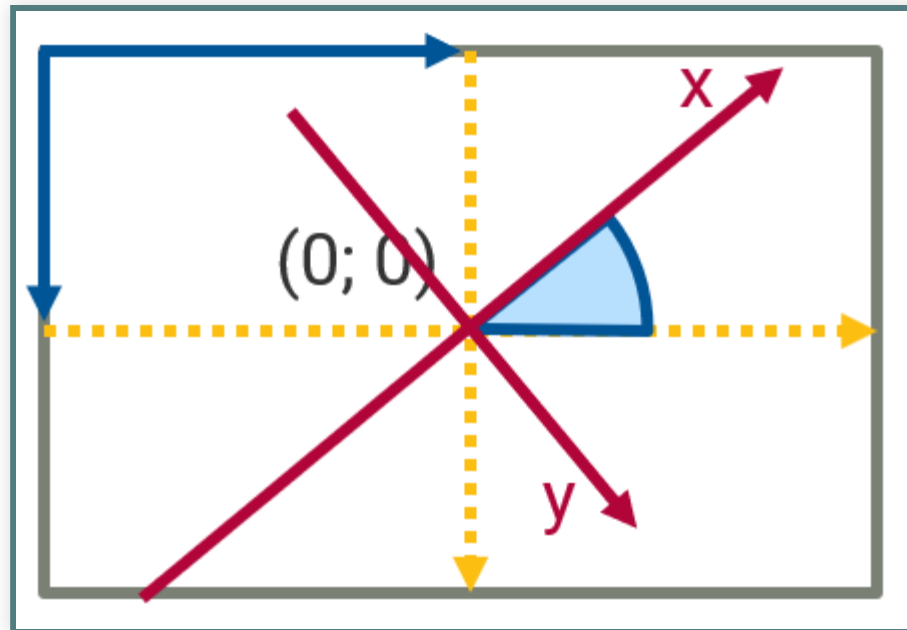
FORGATÁS

```
context.rotate(Math.PI / 4);
```



KOMPLEX TRANSZFORMÁCIÓ

```
context.translate(100, 100);  
context.rotate(-Math.PI / 4);  
// Draw something  
context.rotate(Math.PI / 4);  
context.translate(-100, -100);
```



TRANSZFORMÁCIÓ PÉLDA

```
for (var i = 0; i < 3; i++) {  
  for (var j = 0; j < 3; j++) {  
    ctx.save();  
    ctx.fillStyle = 'rgb(' + (51 * i) + ', ' + (255 - 51 * i) + '  
    ctx.translate(10 + j * 50, 10 + i * 50);  
    ctx.fillRect(0, 0, 25, 25);  
    ctx.restore();  
  }  
}
```





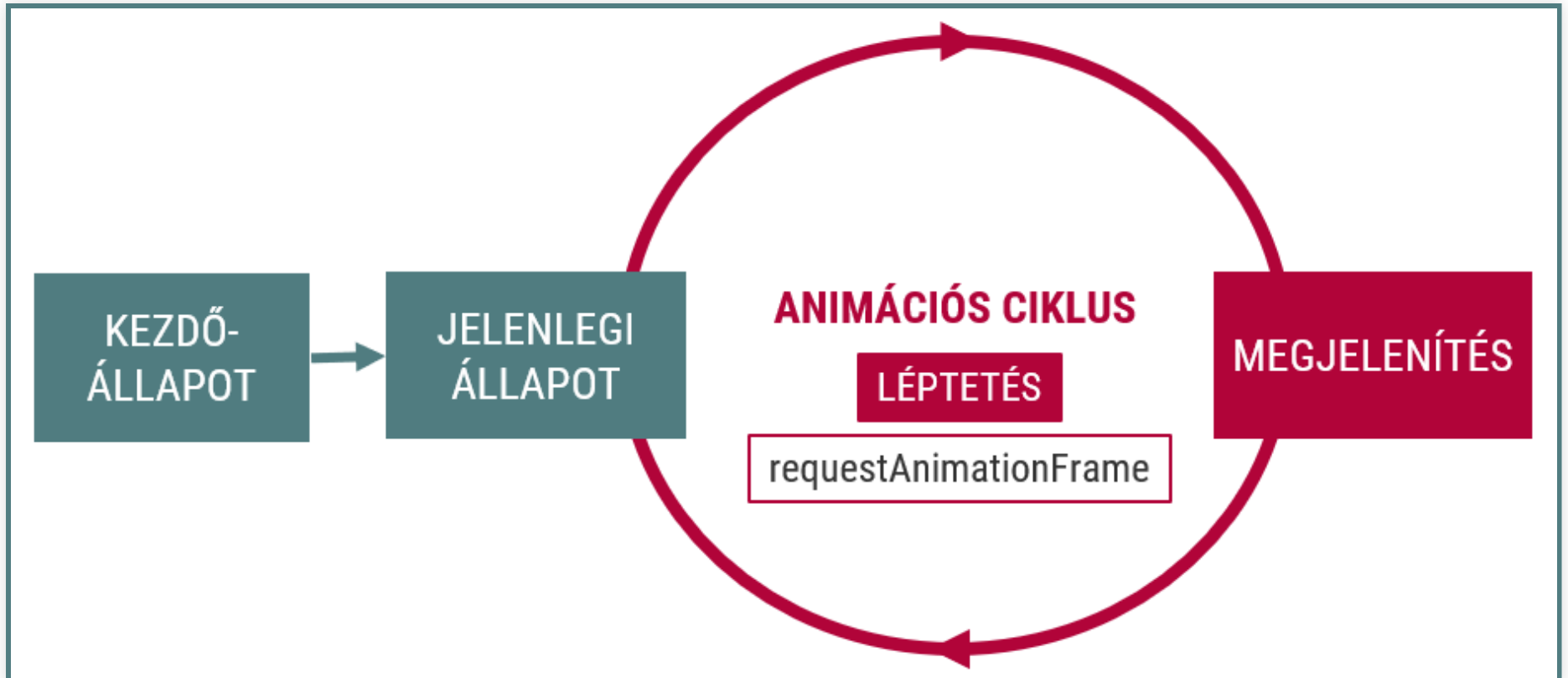
ANIMÁCIÓK

ANIMÁCIÓK

- Alapelv: vásznon lévő kép **gyors** újrarajzolása
 1. Változások rajzolása
 2. Teljes rajzolás
 - canvas törlése
 - rajzolás
- Architektúra
 - Állapottér (adatok)
 - Nézet (rajzolás)



ANIMÁCIÓS CIKLUS



ANIMÁCIÓS CIKLUS

```
const state = /*...*/;

function next() {
  update(); // Update current state
  render(); // Rerender the frame
  requestAnimationFrame(next);
}

next(); // Start the loop

function update() { /* Change app state */ }
function render() { /* Draw app state */ }
```

IDŐ ALAPÚ ANIMÁCIÓ

Az állapot változása az eltelt idő függvénye (pl. fizika)

```
let lastFrameTime = performance.now();
const state = /*...*/;

function next(currentTime = performance.now()) {
  const dt = (currentTime - lastFrameTime) / 1000; // seconds
  lastFrameTime = currentTime;

  update(dt); // Update current state
  render(); // Rerender the frame

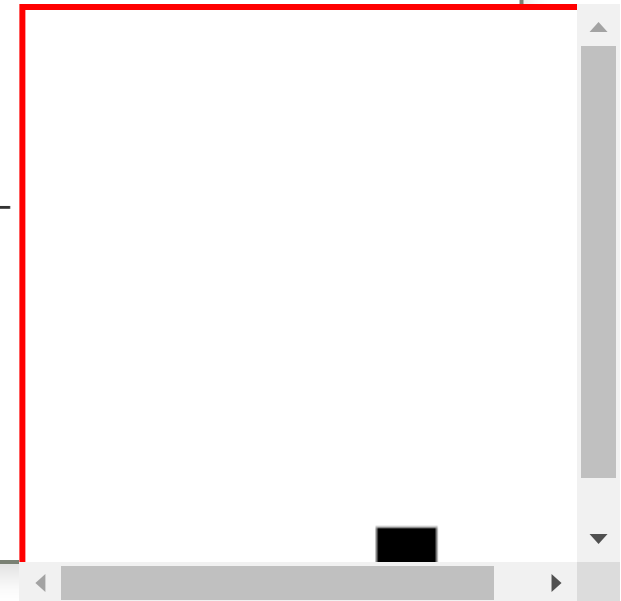
  requestAnimationFrame(next);
}

next(); // Start the loop
```

- $v = s / t$
 - $ds = v * dt$
- $a = v / t$
 - $dv = a * dt$

PÉLDA

```
const box = {  
  x: 50, y: 50,  
  vx: random(50, 200), vy: random(50, 200),  
  width: 30, height: 30,  
}  
  
function update(dt) {  
  box.x += box.vx * dt  
  box.y += box.vy * dt  
  if (box.x + box.width > canvas.width) {  
    box.x = 2 * (canvas.width - box.width) -  
    box.x  
    box.vx *= -1  
  }  
  if (box.x < 0) {  
    box.x *= -1  
    box.vx *= -1  
  }  
  if (box.y + box.height > canvas.height) {  
    box.y = 2 * (canvas.height - box.height) -  
    box.y  
    box.vy *= -1  
  }  
  if (box.y < 0) {  
    box.y *= -1  
    box.vy *= -1  
  }  
}
```



EGYSÉGBE ZÁRÁS

```
class Entity {  
    constructor(canvas, context) {  
        this.canvas = canvas  
        this.context = context  
        /* Initialize */  
    }  
    update(dt) {  
        /* Update properties */  
    }  
    draw() {  
        /* Render to canvas */  
    }  
}
```

PÉLDA

```
class Ball {  
  vx = 500;  
  radius = 20;  
  
  constructor(x = this.radius) {  
    this.x = x;  
  }  
  
  update(dt) {  
    this.x += this.vx * dt / 1000;  
  }  
  
  draw(context) {  
    context.beginPath();  
    context.arc(this.x, this.radius, this.radius, 0, Math.PI *  
    context.fill();  
  }  
}
```



SPRITESHEETEK HASZNÁLATA



SPRITESHEETEK HASZNÁLATA

```
const sprite = {
  currentFrame: 0,
  image: new Image(),
  frameCount: 16,
  frameTime: 0.03,
  elapsedTime: 0,
}

function update(dt) {
  sprite.elapsedTime += dt
  if (sprite.elapsedTime > sprite.frameTime) {
    sprite.currentFrame = (sprite.currentFrame + 1)
    sprite.elapsedTime -= sprite.frameTime
  }
}

function draw() {
  // clear
  ctx.clearRect(0, 0, canvas.width, canvas.height)
```



SPRITE OSZTÁLY

```
class Sprite {
  currentFrame = 0;
  timeSinceLastFrame = 0;
  loopCount = 0;

  constructor({
    image,
    width,
    height,
    spritesPerRow = 1,
    spritesCount = 1,
    frameDuration = 0.03,
    numberOfLoops = Number.POSITIVE_INFINITY,
    isAnimating = true,
  }) {
    this.image = new Image();
    this.image.src = image;
```

ANIMÁCIÓ SPRITE OSZTÁLYAL

```
const sprite = new Sprite({  
  image: "spritesheet.png",  
  width: 128,  
  height: 128,  
  spritesPerRow: 4,  
  spritesCount: 16,  
  frameDuration: 0.03  
});
```

```
let lastFrameTime = performance.now();
```

```
function next() {  
  const currentTime = performance.now();  
  const dt = currentTime - lastFrameTime;  
  lastFrameTime = currentTime;
```

```
context.clearRect(0, 0, canvas.width, canvas.height);
```



SPRITE KITERJESZTÉSE

```
class Runner extends Sprite {  
    direction = 1;  
    x = 0;  
    vx = 200;  
  
    update(dt) {  
        super.update(dt);  
        this.x += this.vx * dt / 1000;  
    }  
  
    render(context) {  
        context.save();  
        context.translate(this.x, 0);  
        context.scale(this.direction, 1);  
        super.render(context, -this.width, 0);  
        context.restore();  
    }  
}
```

SPRITE MOZGATÁSA

```
const runner = new Runner({/*...*/});
let lastFrameTime = performance.now();
function next() { /*...*/ }

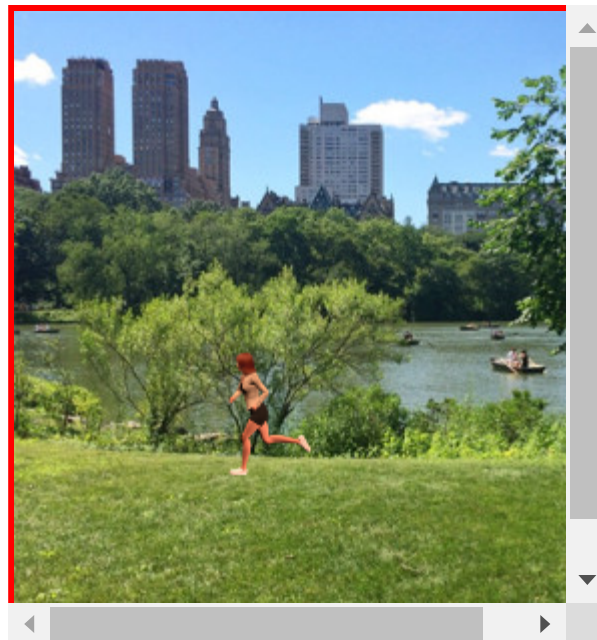
function update(dt) {
  runner.update(dt);

  if (runner.x >= canvas.width || runner.x <= 0) {
    runner.turnAround();
  }
}

function render() {
  runner.render(context);
}
```

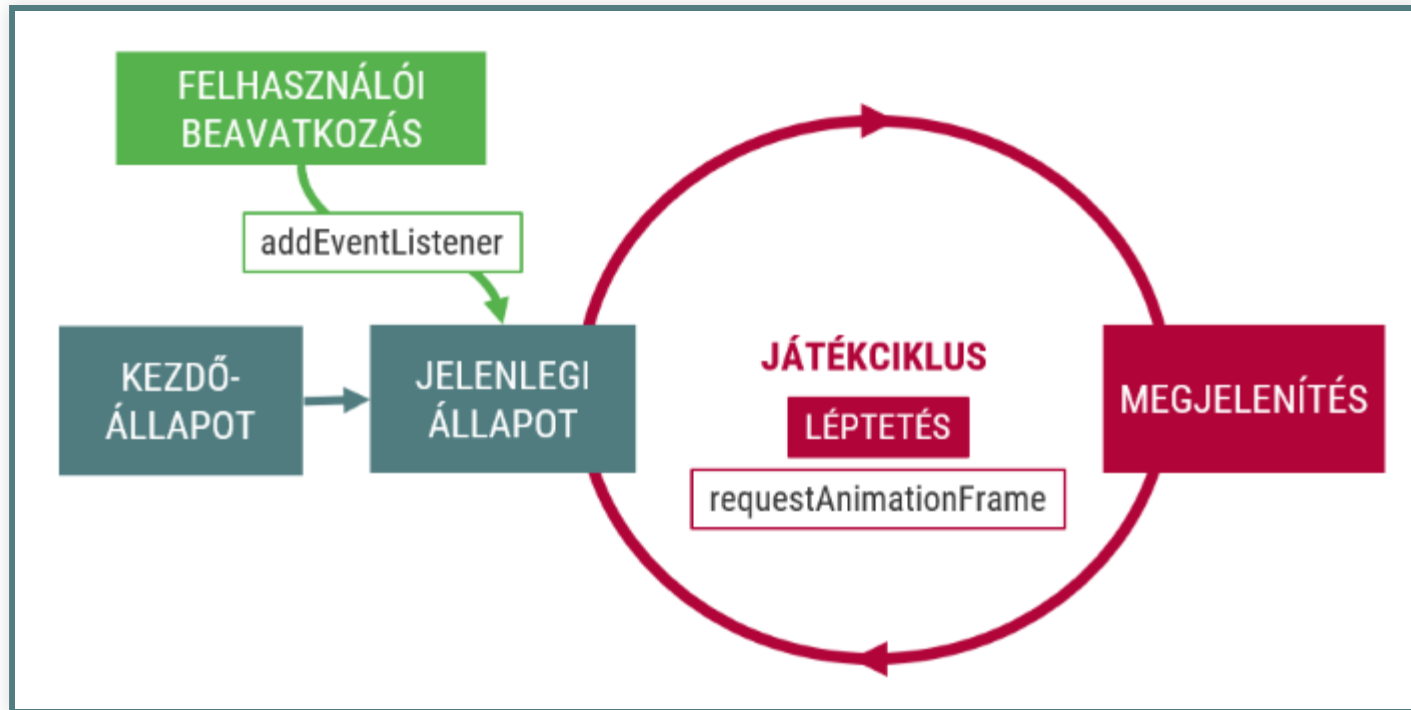


KIS GRAFIKÁVAL...



JÁTÉKOK KÉSZÍTÉSE = INTERAKTÍV ANIMÁCIÓ

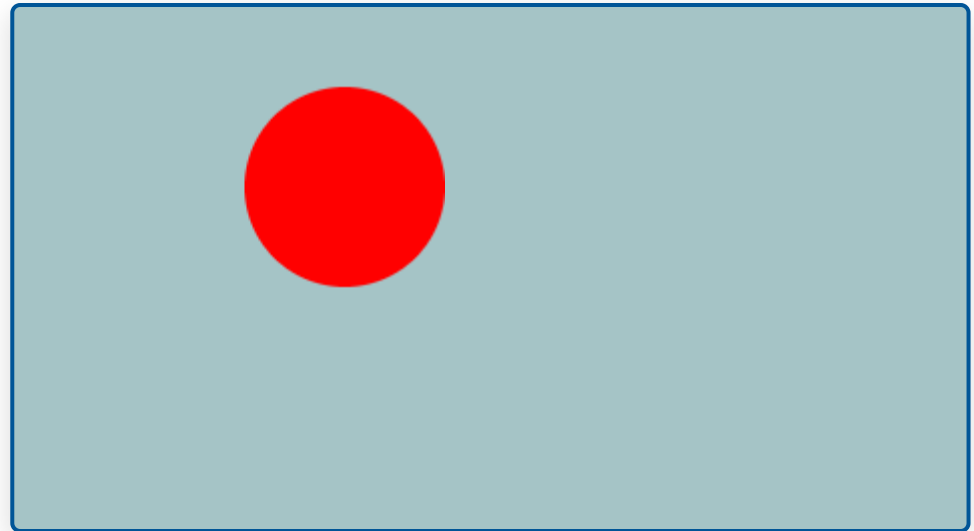
Globális eseménykezelők módosítják az animáció paramétereit



Játék példa (Visnovitz Márton 😊)

INTERAKCIÓ

- `isPointInPath()`
- `isPointInStroke()`
- ⚠ Hit regions
 - `addHitRegion()`
 - `removeHitRegion()`
 - `clearHitRegions()`



CANVAS 3D

A Canvas elemre: WebGL technológiával

```
const canvas = document.querySelector("canvas");  
const webGl = canvas.getContext("webgl");  
webGl.viewport(0, 0, canvas.width, canvas.height);
```

DEMO



Példák

INLINE SVG

```
<html>
  <svg width="300px" height="300px">
    <defs>
      <linearGradient id="myGradient" x1="0%" y1="100%" x2="100%
        <stop offset="5%" stop-color="red"></stop>
        <stop offset="95%" stop-color="blue" stop-opacity="0.5"
      </linearGradient>
    </defs>
    <circle id="myCircle" class="important" cx="50%" cy="50%" r
      fill="url(#myGradient)" onmousedown="alert('hello');"/>
  </svg>
</html>
```



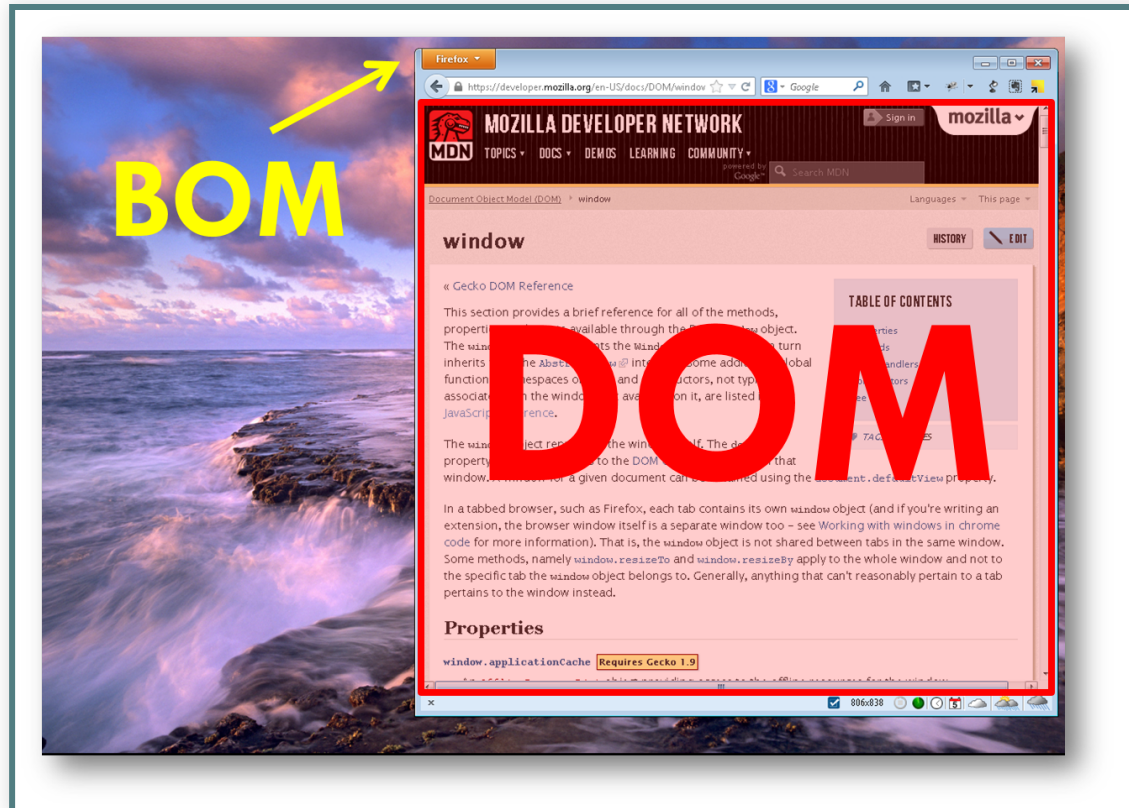
API-K A BÖNGÉSZŐBEN

API-K A BÖNGÉSZŐBEN

Programozási interfészek erőforrások eléréséhez/kezeléséhez

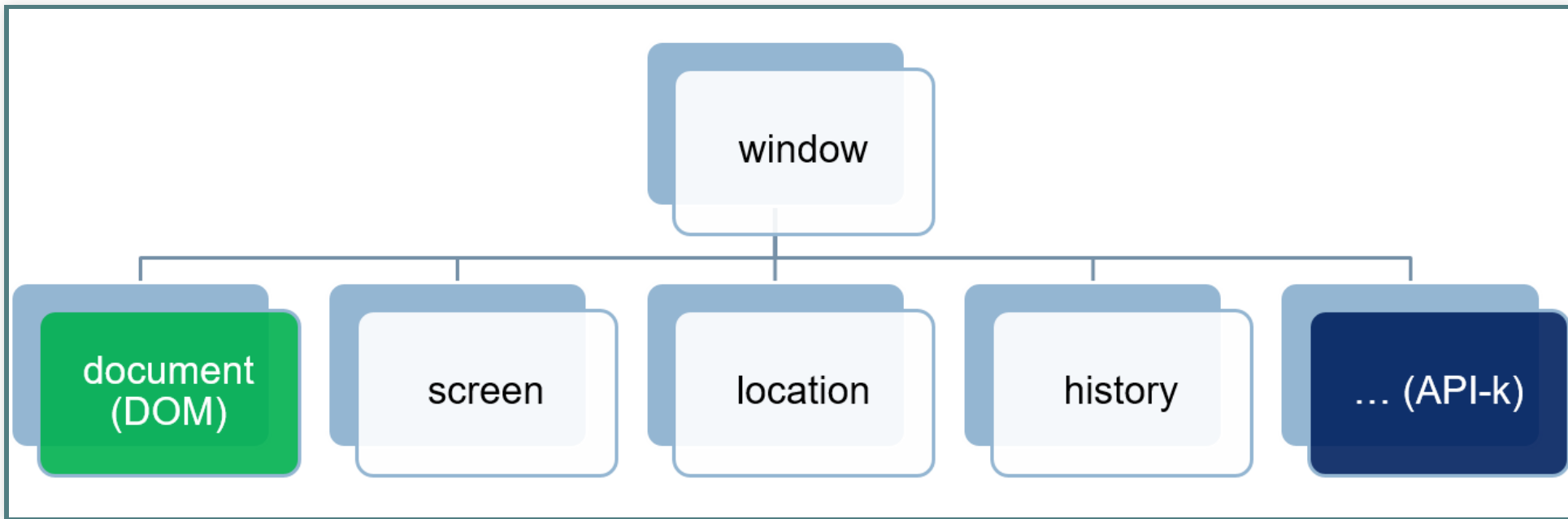
- BOM (Browser Object Model)
- Location API
- Geolocation API
- MediaDevices API
- File API
- ...

A BÖNGÉSZŐABLAK



BOM (**window**)

- DOM-ot tartalmazó ablak (fül), JS kontextusa
- Globális névtér
- Ablakhoz kapcsolódó funkciók
- Sok egyéb API-nak a tárolója
- Browser Object Model (BOM)
- Referencia



GLOBALIS NÉVTÉR

```
function hello() {  
  console.log('Hello');  
}  
window.hello()  
  
// Unintentional global variable  
function sideEffect() {  
  let aVeryBigVariable  
  aVeryBigvariable = 12; // window.aVeryBigvariable  
}  
sideEffect()  
console.log(aVeryBigvariable);  
console.log(window.aVeryBigvariable);  
  
// strict mode  
function sideEffectStrict() {  
  "use strict"
```

ÚJ ABLAK

- `window.open()`, `window.close()`
- `opener`
- Az új ablak referenciája egy `window` objektum

```
const options = "resizable,width=800,height=600,scrollbars=yes"
const elte = window.open("http://www.elte.hu", "ELTE", options)
elte.resizeTo(400, 200);
elte.document.querySelector("p");
elte.opener; // window
elte.close();
```

Open new window

LOCATION API

- A címsor tartalmának olvasása
- Az egyes részek külön kezelhezők
- `hash`, `host`, `hostname`, `href`, `origin`, `pathname`, `port`, `protocol`, `search`, `username`, `password`

```
// http://example.com:8080/page.html?name=value#anchor
location.href;           // az egész URL
location.host;           // "example.com:8080"
location.hostname;       // "example.com"
location.origin;         // "http://example.com:8080"
location.pathname;       // "/page.html"
location.port;           // "8080"
location.protocol;       // "http:"
location.search;         // "?name=value"
location.hash;           // "#anchor"
```

LOCATION API

■ URL módosítása

- `assign(newUrl)`: új oldal betöltése
- `replace(newUrl)`: jelenlegi felülírása
- `reload()`: újratöltés

■ Események

- `hashchange`: az URL fragment megváltozása

```
window.location = "http://www.elte.hu";  
window.location.href = "http://www.elte.hu";  
window.location.assign("http://www.elte.hu");  
window.location.replace("http://www.elte.hu");  
window.location.reload();
```

URLSearchParams

A `location.search` paraméter feldolgozására

```
const searchParams = new URLSearchParams(location.search);

searchParams.has("paramName");
searchParams.get("paramName");
searchParams.getAll("paramName");
searchParams.set("paramName");
searchParams.append("paramName", "paramValue");
searchParams.delete("paramName");
```

a =

b =

Küldve: a = b =

Referencia

HISTORY

Közlekedés az előzményekben

- `window.history`
 - `back()`
 - `forward()`
 - `go(n)`

```
window.history.back();  
window.history.forward();  
window.history.go(-3);
```

HISTORY

Módosítás

- `window.history.pushState(stateObj, name, url)`
- `window.history.replaceState(stateObj, name, url)`
- `window.onpopstate` esemény

```
window.onpopstate = function(e) {  
    console.log("location: " + document.location + ", state: " +  
};  
  
history.pushState({page: 1}, "title 1", "?page=1");  
history.pushState({page: 2}, "title 2", "?page=2");  
history.replaceState({page: 3}, "title 3", "?page=3");  
history.back(); // logs "location: http://example.com/example.h  
history.back(); // logs "location: http://example.com/example.h  
history.go(2);  // logs "location: http://example.com/example.h
```

DEMO

IFRAME PROGRAMOZÁS

- Támogatott
- Ablak az ablakon belül
- Szeparált környezet
 - sandboxing
 - aszinkron kommunikáció
- Elérés
 - `window.parent`
 - `iframe.contentWindow`
 - `iframe.contentWindow.document` /
`iframe.contentDocument`
- Same Origin Policy

POSTMESSAGE

- Felügyelt kommunikáció ablakok között
- `postMessage()`, `message` esemény

```
<iframe src="http://localhost:8081/window.postMessage.iframe.ht
```

```
// parent window
window.addEventListener('load', function () {
  const iframe = document.querySelector('iframe')
  iframe.contentWindow.postMessage("message", "http://localho
})
window.addEventListener('message', function (e) {
  console.log(e.origin, e.data)
})
```

```
// iframe
window.addEventListener('message', function (e) {
  if (e.origin === "http://localhost:8080") {
    console.log(e.data)
    window.parent.postMessage('message back', 'http://local
  }
})
```

Kell két http-server: 8080, 8081

TOVÁBBI WINDOW TULAJDONSÁGOK

■ Tulajdonság

- `name`
- `fullscreen`

■ Események

- `load`, `unload`
- `abort`, `close`
- `contextmenu`
- `resize`, `scroll`,
`select`
- `storage`
- `copy`, `cut`, `paste`

■ Metódusok

- `alert`, `confirm`, `prompt`
- `atob`, `btoa` (base64
kódolás)
- `matchMedia`
- `print`
- `postMessage`
- `stop`
- `fetch`

WEB WORKERS

- Valódi többszálúság
- Kommunikáció: üzenetek/események

```
//főszál
const worker = new Worker('masik.js');
worker.onmessage = function(e) {
  console.log(e.data);
};
worker.postMessage('valami');

//masik.js
self.onmessage = function(e) {
  self.postMessage("Kapott adat: " + e.data);
};
```

DRAG AND DROP

- Vonszolható objektum: `draggable="true"`
- `dragstart`, `dragenter`, `dragover`, `drop`, stb. események
- `event.dataTransfer`
 - `setData`
 - `setDragImage`
 - `effectAllowed`
 - `dropEffect`
- <http://html5demos.com/drag>

DRAG AND DROP

Példa

1. `draggable` attribútum
2. `dragstart` esemény: drag adatok tárolása
3. `dragover` esemény: dropzone kijelölése
4. `dragleave` esemény: dropzone elhagyása
5. `drop` esemény: ejtés
6. `dragend` esemény: drag adatok alaphelyzetbe

TOVÁBBI HTML 5 JAVASCRIPT API

- Média
 - Fullscreen
- Offline alkalmazások
 - Application Cache
 - window online/offline eseményei
 - navigator.online
- Contenteditable
- Web sockets, Server Sent Events
- Geolocation
- File API (desktop), Filesystem



MÉDIAELEMENK VEZÉRLÉSE

AUDIO

- Hangállományok lejátszása
- `HTMLAudioElement`

```
<audio src="horn.wav" id="audio1" controls></audio>
```

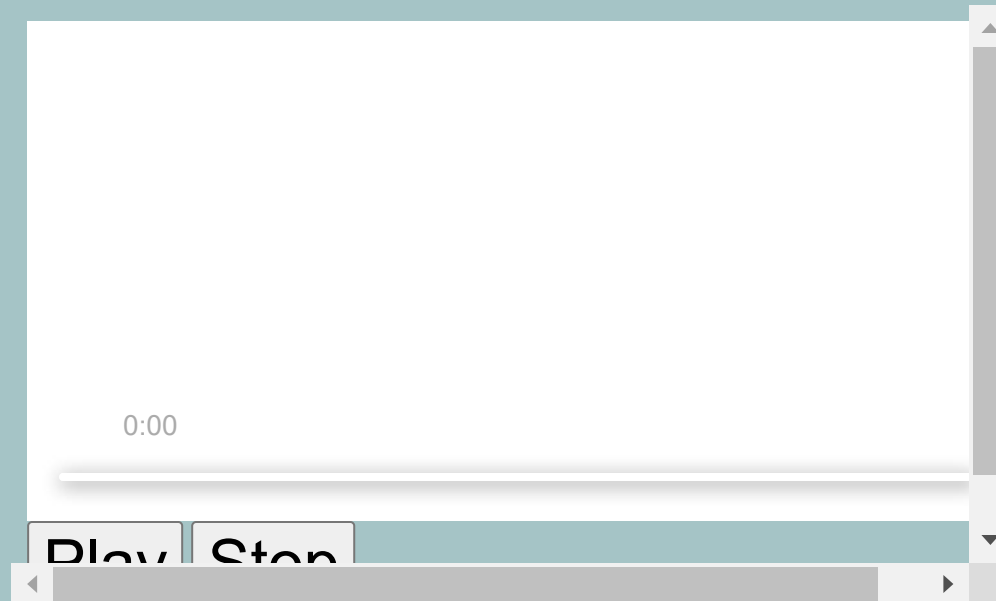
```
// meglévő elem  
document.querySelector('audio').play();  
  
// in-memory elem  
const audio = document.createElement('audio');  
audio.src = 'horn.wav';  
audio.play();
```

0:00 / 0:00

Play Stop

VIDEO

- HTML `<video>` tag
- `HTMLVideoElement`



VIDEO

```
const canvas = document.querySelector('canvas')
const ctx = canvas.getContext('2d')
const video = document.querySelector('video')
const mainloop = function() {
  window.requestAnimationFrame(mainloop);
  ctx.drawImage(video, 0, 0, canvas.width, canvas.height)
}
mainloop()
```

0:00

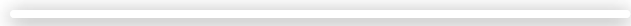


VIDEO

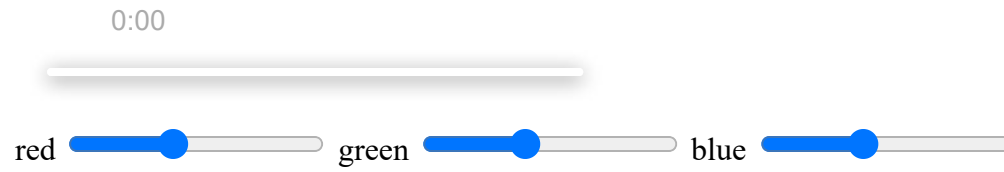
```
let deg = 0;
const mainloop = function() {
  window.requestAnimationFrame(mainloop);

  deg += 0.01
  ctx.clearRect(0, 0, canvas.width, canvas.height)
  ctx.save()
  ctx.translate(canvas.width / 2, canvas.height / 2)
  ctx.scale(0.5, 0.5)
  ctx.rotate(deg)
  ctx.drawImage(video, -canvas.width / 2, -canvas.height / 2, c
  ctx.restore()
}
```

0:00



VIDEO



VIDEO

```
const canvas = document.querySelector('canvas')
const ctx = canvas.getContext('2d')
const image = document.querySelector('img')
const video = document.querySelector('video')
const red = document.querySelector('#red')
const green = document.querySelector('#green')
const blue = document.querySelector('#blue')

navigator.mediaDevices.getUserMedia({video: true})
  .then(function(mediaStream) {
    image.src = mediaStream
    video.srcObject = mediaStream
    video.addEventListener('loadeddata', function () {
      video.play()
      mainloop()
    })
  })
})
```


ÖSSZEFOGLALÁS

- Canvas
- Animációk
- `window` (BOM)
- JavaScript API-k
- Médiaelemek programozása

